Laboratório Funcional · Aula 12

Gustavo Cardoso · RA 240111752 · Turma A
Gerado em 16/10/2026, 20:22:01 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora()
Essa função é impura porque utiliza datetime.now(), que depende do horário atual. Assim, chamadas diferentes podem retornar resultados diferentes mesmo sem receber argumentos.
registra(msg)
Essa função é impura porque utiliza print(msg) para exibir uma mensagem na tela. Isso gera um efeito colateral, pois altera o que é mostrado ao usuário, em vez de apenas retornar um resultado.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2
4: 14
7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

não respondido

```python
# Prática 1 - Funções puras e formas funcionais (Sebesta 15.2 e 15.3, p. 633-637)
# Cole em https://onecompiler.com/python
from functools import reduce

contador = 0

def proximo_id(prefixo):            # IMPURA: depende de um estado que muda
    global contador
    contador += 1
    return f"{prefixo}-{contador}"

def cubo(x):                        # PURA: só depende do parâmetro
    return x * x * x

print("1:", proximo_id("A"), proximo_id("A"))
print("2:", cubo(2) + cubo(2) == 2 * cubo(2))
print("3:", (lambda x: x * x * x)(2))           # a expressão lambda da p. 635

def compor(f, g):                   # composição: compor(f, g)(x) = f(g(x))
    return lambda x: f(g(x))

f = lambda x: x + 2
g = lambda x: 3 * x
print("4:", compor(f, g)(4))
print("5:", list(map(lambda x: x * x, [2, 3, 4])))      # aplicar-a-todos (p. 636)
print("6:", reduce(lambda acc, x: acc + x, [4, 9, 16], 0))

notas = [7, 3, 9]
```

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

```python
# Prática 1 - Funções puras e formas funcionais (Sebesta 15.2 e 15.3, p. 633-637)
# Cole em https://onecompiler.com/python
from functools import reduce

contador = 0

def proximo_id(prefixo):            # IMPURA: depende de um estado que muda
    global contador
    contador += 1
    return f"{prefixo}-{contador}"

def cubo(x):                        # PURA: só depende do parâmetro
    return x * x * x

print("1:", proximo_id("A"), proximo_id("A"))
print("2:", cubo(2) + cubo(2) == 2 * cubo(2))
print("3:", (lambda x: x * x * x)(2))           # a expressão lambda da p. 635

def compor(g, f):                   # composição: compor(f, g)(x) = f(g(x))
    return lambda x: f(g(x))

f = lambda x: x + 2
g = lambda x: 3 * x
print("4:", compor(f, g)(4))
print("5:", list(map(lambda x: x * x, [2, 3, 4])))      # aplicar-a-todos (p. 636)
print("6:", reduce(lambda acc, x: acc + x, [4, 9, 16], 0))

notas = [7, 3, 9]
```

```
1: A-1 A-2
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função é impura porque utiliza a variável global contador, que é incrementada a cada chamada. Assim, mesmo passando "A" como argumento, ela retorna valores diferentes, como A-1 e A-2. Isso dificulta os testes porque o resultado depende do estado anterior do programa. Para testar a função corretamente, seria necessário controlar ou reiniciar o contador, tornando os testes mais complexos e menos previsíveis.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4)
g(4) = 3 x 4 = 12

f(12) = 12 + 2 = 14
R: 14

compor(g, f)(4)
f(4) = 4 + 2 = 6
g(6) = 3 x 6 = 18
R: 18

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria e retorna uma nova lista ordenada, preservando a lista original. Já notas.sort() ordena a própria lista, modificando-a diretamente. O método que segue o estilo funcional é sorted(notas), porque evita modificar os dados originais e produz um novo resultado. Isso torna o comportamento mais previsível e facilita os testes.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

22

Previsão: (cdr '(A B C))

14

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

não respondido

```
; Prática 2 - Scheme: o essencial (Sebesta 15.5, p. 641-650)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(mostra (+ 3 7 10 2))              ; notação prefixa: o operador vem primeiro
(mostra (* 2 (+ 3 4)))

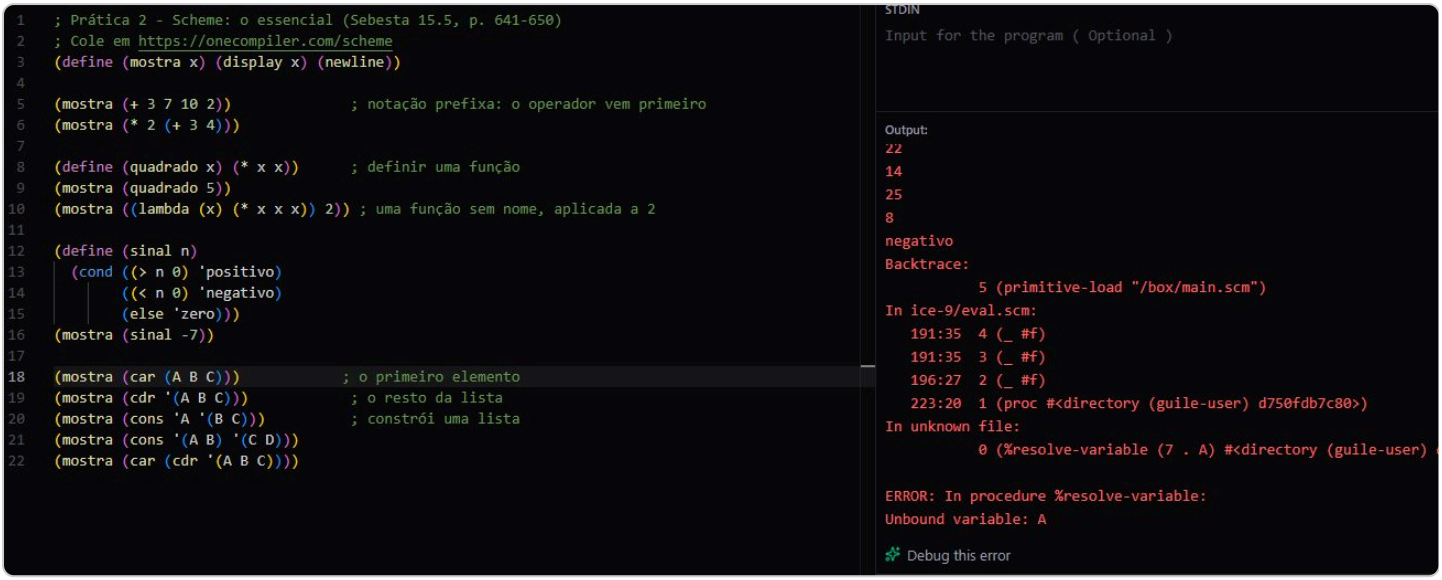
(define (quadrado x) (* x x))      ; definir uma função
(mostra (quadrado 5))
(mostra ((lambda (x) (* x x x)) 2)) ; uma função sem nome, aplicada a 2

(define (sinal n)
  (cond ((> n 0) 'positivo)
        ((< n 0) 'negativo)
        (else 'zero)))
(mostra (sinal -7))

(mostra (car '(A B C)))            ; o primeiro elemento
(mostra (cdr '(A B C)))            ; o resto da lista
(mostra (cons 'A '(B C)))          ; constrói uma lista
(mostra (cons '(A B) '(C D)))
(mostra (car (cdr '(A B C))))
```

```
STDIN
Input for the

Output:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```



```
; Prática 2 - Scheme: o essencial (Sebesta 15.5, p. 641-650)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(mostra (+ 3 7 10 2))              ; notação prefixa: o operador vem primeiro
(mostra (* 2 (+ 3 4)))

(define (quadrado x) (* x x))      ; definir uma função
(mostra (quadrado 5))
(mostra ((lambda (x) (* x x x)) 2)) ; uma função sem nome, aplicada a 2

(define (sinal n)
  (cond ((> n 0) 'positivo)
        ((< n 0) 'negativo)
        (else 'zero)))
(mostra (sinal -7))

(mostra (car (A B C)))             ; o primeiro elemento
(mostra (cdr '(A B C)))            ; o resto da lista
(mostra (cons 'A '(B C)))          ; constrói uma lista
(mostra (cons '(A B) '(C D)))
(mostra (car (cdr '(A B C))))
```

```
STDIN
Input for the program ( Optional )

Output:
22
14
25
8
negativo
Backtrace:
          5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) d750fdb7c80>)
In unknown file:
          0 (%resolve-variable (7 . A) #<directory (guile-user)

ERROR: In procedure %resolve-variable:
Unbound variable: A

Debug this error
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo serve para impedir que uma expressão seja avaliada como uma chamada de função. Ele indica que o conteúdo deve ser tratado como um dado literal.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

não respondido

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Vai funcionar

Saída do Scheme

não respondido

```scheme
; Prática 3a - Recursão e recursão em cauda (Sebesta 15.5.10-15.5.12, p. 651-656)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(define (membro atm lista)              ; o member do livro (p. 651)
  (cond ((null? lista) #f)              ; 1) lista vazia
        ((eq? atm (car lista)) #t)      ; 2) olha o primeiro
        (else (membro atm (cdr lista))))) ; 3) procura no resto
(mostra (membro 'B '(A B C)))
(mostra (membro 'Z '(A B C)))

(define (fatorial n)                    ; a multiplicação fica para a VOLTA
  (if (<= n 1) 1 (* n (fatorial (- n 1)))))

(define (facthelper n parcial)          ; em cauda: a conta vai no parâmetro (p. 656)
  (if (<= n 1) parcial (facthelper (- n 1) (* n parcial))))

(mostra (fatorial 20))
(mostra (facthelper 20 1))

(define (conta-ate n acc)               ; um milhão de chamadas em cauda
  (if (= n 0) acc (conta-ate (- n 1) (+ acc 1))))
(mostra (conta-ate 1000000 0))
```

```
STDIN
Input for the program

Output:
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

não respondido

```python
# Prática 3b - A mesma recursão em cauda, agora em Python
# Cole em https://onecompiler.com/python
def conta_ate(n, acc):
    if n == 0:
        return acc
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação

print(conta_ate(500, 0))
print(conta_ate(1_000_000, 0))
```

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última
operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última
operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última
operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, cresce a quantidade de registros de ativação na pilha, pois cada chamada precisa esperar a anterior retornar para concluir a operação pendente, como a multiplicação do fatorial.
Na recursão em cauda, não há operações pendentes após a chamada recursiva. Por isso, uma linguagem que otimiza chamadas em cauda pode reutilizar o registro de ativação, mantendo o uso de pilha constante.

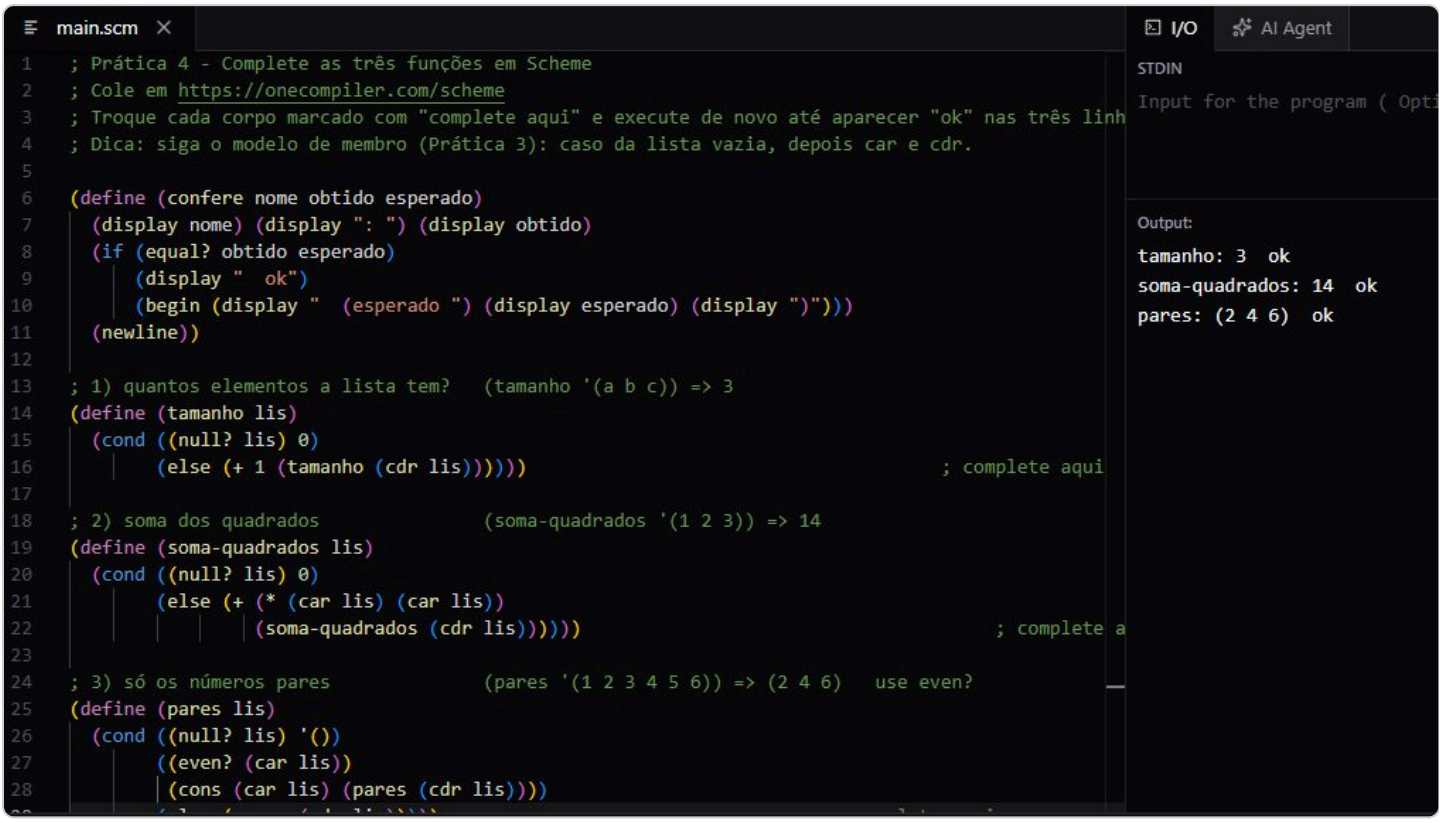
3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme utilizado na atividade otimiza a recursão em cauda, permitindo que a função conta-ate faça um milhão de chamadas sem precisar acumular um registro de ativação para cada uma. O Python, por outro lado, não realiza otimização de chamadas em cauda. Mesmo quando a chamada recursiva é a última operação, cada chamada continua ocupando espaço na pilha de execução. Como o Python possui um limite de profundidade de recursão, a execução termina com RecursionError antes de chegar a um milhão.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

não respondido




```
; Prática 4 - Complete as três funções em Scheme
; Cole em https://onecompiler.com/scheme
; Troque cada corpo marcado com "complete aqui" e execute de novo até aparecer "ok" nas três linh
; Dica: siga o modelo de membro (Prática 3): caso da lista vazia, depois car e cdr.

(define (confere nome obtido esperado)
  (display nome) (display ": ") (display obtido)
  (if (equal? obtido esperado)
      (display "  ok")
      (begin (display "  (esperado ") (display esperado) (display ")")))
  (newline))

; 1) quantos elementos a lista tem?   (tamanho '(a b c)) => 3
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))                    ; complete aqui

; 2) soma dos quadrados               (soma-quadrados '(1 2 3)) => 14
(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis))
                 (soma-quadrados (cdr lis))))))                    ; complete a

; 3) só os números pares              (pares '(1 2 3 4 5 6)) => (2 4 6)   use even?
(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))

(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis))
                 (soma-quadrados (cdr lis))))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como a recursão percorre a lista até chegar ao final. usei null? para verificar se a lista estava vazia e definir o caso-base.

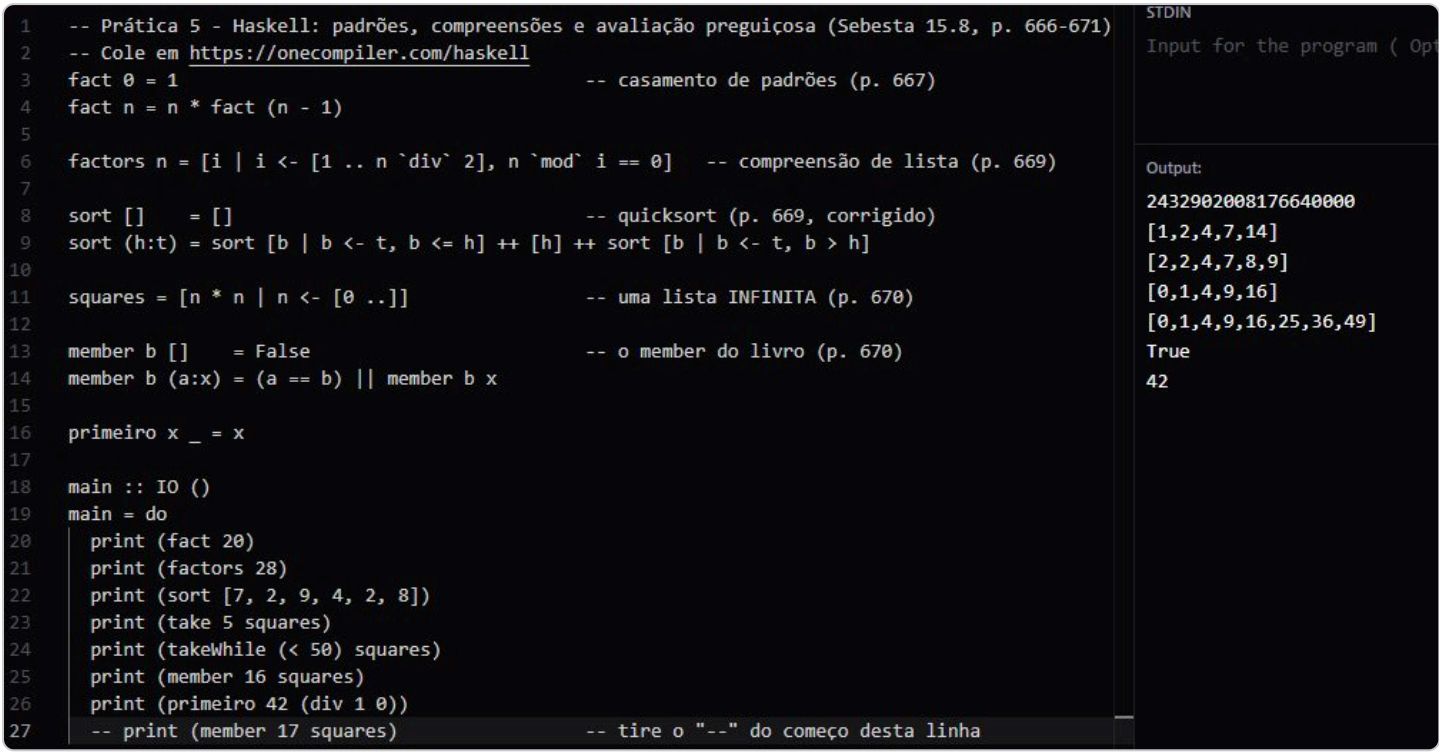
Prática 5: Haskell: padrões, compreensões e preguiça

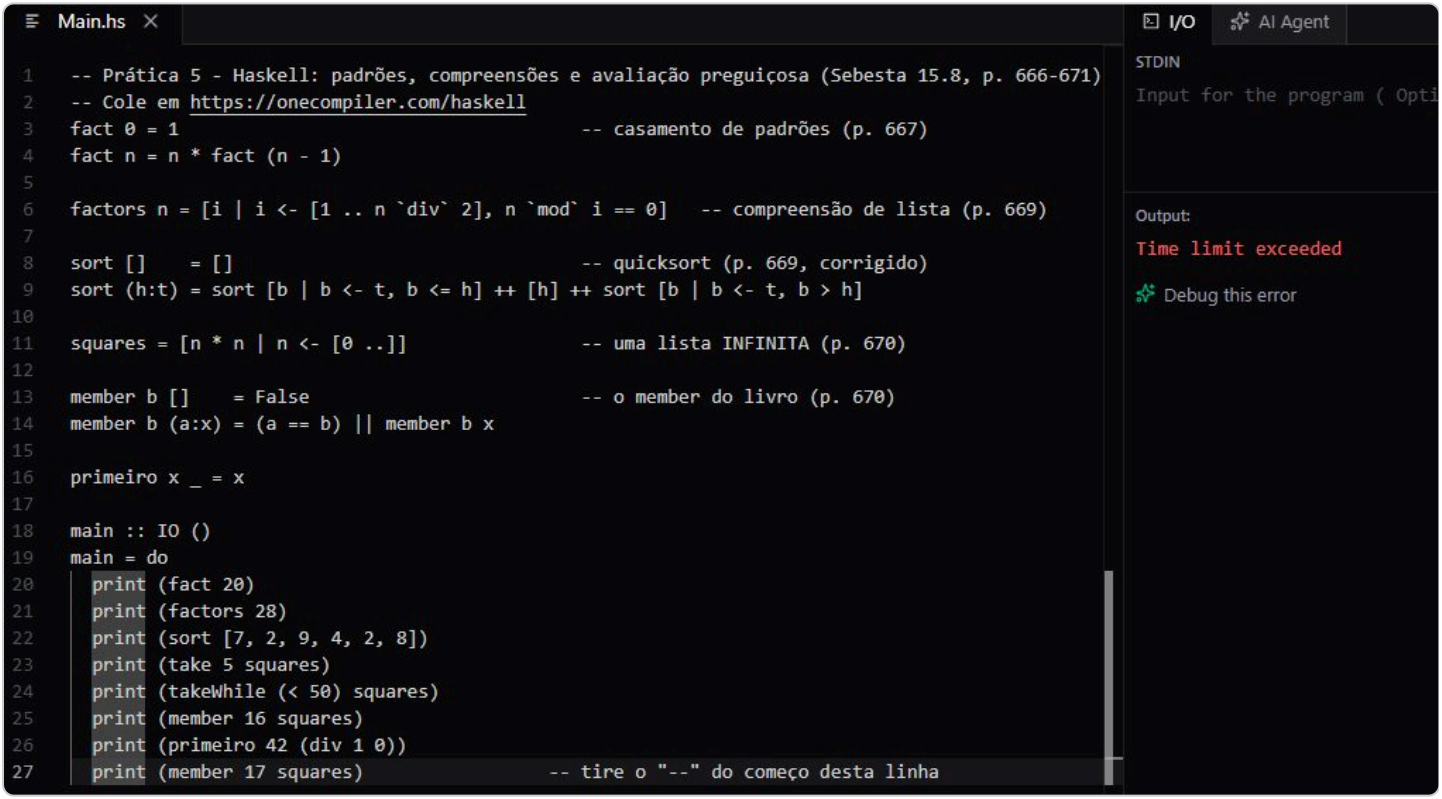
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

[0, 1, 4, 9, 16], 42

Saídas (original e com member 17)

não respondido





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

calculando apenas os elementos que são necessários. Assim, take 5 squares calcula somente os cinco primeiros elementos da lista infinita, imprimindo [0,1,4,9,16].

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

A busca por 17 nunca termina porque squares contém os quadrados perfeitos em ordem crescente: 0, 1, 4, 9, 16, 25, 36.... Como 17 não aparece na lista, a função continua procurando indefinidamente. Já member 16 squares termina porque encontra o número 16 e retorna True. Eu corrigiria verificando se o elemento atual é maior que o valor procurado como a lista está em ordem crescente nesse caso já podemos retornar False e encerrar a busca

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

15 e [5,10,15]

Saída do JavaScript

não respondido

```javascript
// Prática 6a - Fechamentos e currying em JavaScript (Sebesta 15.7 e 15.10, p. 6
// Cole em https://onecompiler.com/nodejs
const somador = (n) => (x) => x + n;       // devolve uma função que "lembra" o
const mais10 = somador(10);
console.log(mais10(5));

const vezes = (a) => (b) => a * b;         // currying: uma função de um parâmet
console.log([1, 2, 3].map(vezes(5)));

const soma = [1, 2, 3, 4, 5, 6]
  .filter((n) => n % 2 === 0)
  .map((n) => n * n)
  .reduce((acc, n) => acc + n, 0);
console.log(soma);

const original = Object.freeze({ nome: "Ana", nota: 7 });
const corrigido = { ...original, nota: 8 };   // um valor NOVO; o original não m
console.log(original, corrigido);
```

```
Console
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

não respondido

```java
// Prática 6b - Lambdas e Streams em Java (Sebesta 15.10, p. 676)
// Cole em https://onecompiler.com/java
import java.util.*;
import java.util.function.*;

public class Main {
    record Produto(String nome, double preco) {}

    public static void main(String[] args) {
        Function<Integer, Integer> f = x -> x + 2;
        Function<Integer, Integer> g = x -> 3 * x;
        System.out.println(f.compose(g).apply(4));     // f(g(4))
        System.out.println(f.andThen(g).apply(4));     // g(f(4))

        List<Produto> produtos = List.of(new Produto("Teclado", 150), new Produto("Mouse", 80),
                                         new Produto("Monitor", 900), new Produto("Cabo", 35));
        double caros = produtos.stream()
            .filter(p -> p.preco() > 100)
            .mapToDouble(Produto::preco)
            .sum();
        System.out.println(caros);
        System.out.println(produtos.stream().map(p -> p.nome().toUpperCase()).sorted().toList());
    }
}
```

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Isso acontece por causa de um fechamento (closure). Quando somador(10) é executada, ela devolve outra função que mantém acesso ao ambiente em que foi criada, incluindo o valor n = 10.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados são diferentes porque a ordem de aplicação das funções muda. O método que corresponde a compor(f, g) da Prática 1 é f.compose(g), pois a composição definida naquela prática calcula \(f(g(x))\): primeiro g, depois f.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();

        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

não respondido

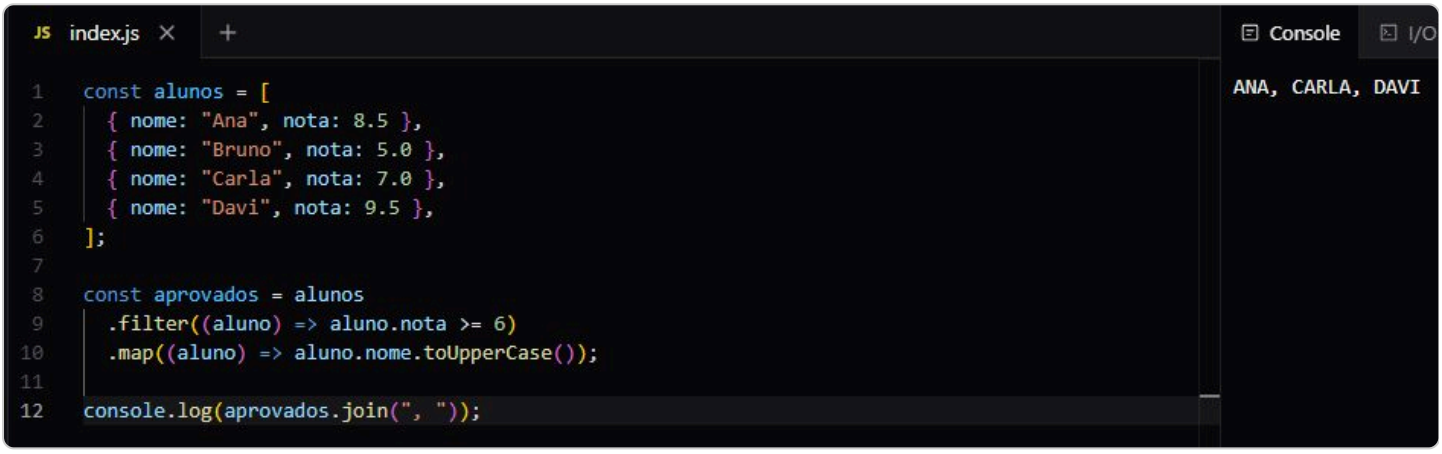
```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n -> n % 2 == 0)
            .mapToInt(n -> n * n)
            .sum();

        System.out.println(soma);
    }
}
```

```
244
```



7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler quando queremos entender rapidamente o que o programa faz, pois separa as operações de filtragem, transformação e cálculo. Já a versão com laço pode ser mais fácil de depurar passo a passo, porque permite acompanhar cada iteração e verificar os valores intermediários. Portanto, a melhor escolha depende da situação: a programação funcional facilita a visualização das operações, enquanto os laços podem facilitar a investigação de erros durante a execução.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão funcional em Haskell pode ser mais fácil de entender porque expressa diretamente as operações de gerar os números, elevar cada um ao cubo e somar os resultados, sem precisar controlar explicitamente um laço e uma variável acumuladora. Eu aceitaria um possível custo maior de execução em sistemas de análise de dados, cálculos científicos ou aplicações em que a clareza e a manutenção do código sejam importantes. Porém, evitaria esse custo em sistemas de tempo real, jogos ou aplicações de alto desempenho, nos quais a velocidade de execução é essencial.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Novamente a dificuldade dela, considerando a minha vontade, nada contra o formato, mas juntando o fator de ser Sexta-feira e que mesmo eu parando para ler e tentar entender tudo, percebo que só demoraria muito (em tempo de aula) conceitos que acharia mais fácil compreender com o professor explicando.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O problema é b <- h. O símbolo h representa o pivô, que é um único elemento, e não uma lista da qual b possa ser retirado. O correto é usar a condição b <= h, selecionando os elementos menores ou iguais ao pivô.
A mensagem do compilador indica um erro de tipos: ele esperava uma lista para a expressão usada como fonte do gerador b <- ..., mas encontrou um valor do tipo do elemento, como um número inteiro (Int). Isso acontece porque h é um elemento, não uma lista.